

TP: Programmation Système en C sous Unix

Exercice 1 :

Ecrire un code qui permet de sauvegarder des données dans un fichier avant de se terminer lors d'une interruption par Ctrl-C dans le terminal. Le fichier sera remplacé s'il n'existe pas, sinon il sera créé.

Le principe est de modifier le comportement lors de la réception du signal SIGINT.

Exercice 2 :

Décrire précisément ce que fait le programme suivant :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int
main (void)
{
    int    tube[2];
    char * buffer = "AZERTYUIOP";

    fprintf(stdout, "Création tube \n");
    if (pipe(tube) != 0) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
    fprintf(stdout, "Fermeture sortie \n");
    close(tube[0]);
    fprintf(stdout, "Écriture dans tube \n");
    if (write(tube[1], buffer, strlen(buffer)) != strlen(buffer)) {
        perror("write");
        exit(EXIT_FAILURE);
    }
    fprintf(stdout, "Fin du programme \n");
    return EXIT_SUCCESS;
}
```

En cas d'erreur, le code fait objet de quel signal ?

Exercice 3 :

Ecrire un programme en C qui permet une transmission bidirectionnelle d'informations entre un processus père et un de ses fils via des tubes : le père envoie 5 entiers au fils qui les affiche et renvoie ces entiers multipliés par 2. Le père affiche ces doubles :

- on crée deux tubes p1 et p2 pour faire communiquer les deux processus :
 - le père a accès en écriture sur p1 et en lecture sur p2,

- le fils a accès en écriture sur p2 et en lecture sur p1.
- Le gestionnaire permet la lecture du nombre sur le tube P1, le calcul du double, puis l'écriture sur le tube P2